Benchmark di inferenza LLM su hardware consumer

LFM2.5-1.2B su un portatile del 2021

Misurazioni reali, non stime. Throughput di un modello linguistico da 1.2 miliardi di parametri eseguito interamente su un portatile con 8 GB di RAM (Intel i5-1135G7, Iris Xe, Windows 10). Nessun cloud, nessuna GPU dedicata, nessun server.

Autore: andiekobbietks (AFJ / AK Build)

Data: 2026-11-03

Hardware: ASUS VivoBook, Intel i5-1135G7, Intel Iris Xe, 8 GB RAM, Windows 10

Modello: LiquidAI LFM2.5-1.2B-Instruct

Indice

1. Introduzione e obiettivo
2. Hardware e ambiente
3. Metodologia
4. Risultati
5. Risultati chiave
6. Budget RAM e tab del browser
7. Guida alla riproduzione
8. Issue upstream e fix
9. Limitazioni

Appendice A: Script completi

Appendice B: Ambiente

1. Introduzione e obiettivo

L'affermazione da verificare: "LFM2.5-1.2B gira a 40-70 token al secondo su un Intel i5-1135G7 con Iris Xe via Vulkan, out-of-the-box." Questo documento documenta la verifica di quell'affermazione su hardware consumer reale, un ASUS VivoBook del 2021 con 8 GB di RAM, e spiega come ogni numero è stato misurato, quale codice è stato usato, e cosa i risultati significano per il deployment di AI on-device.

La maggior parte dei benchmark di edge LLM usa GPU dedicate (RTX 4090) o CPU di fascia alta (32+ GB RAM). Questo lavoro testa lo scenario realistico: un portatile di 5 anni con 8 GB e grafica integrata. Il risultato che la CPU (AVX2) batte la GPU integrata (Vulkan/OpenVINO) per modelli piccoli è controintuitivo e raramente documentato.

Questo documento funge sia da report di benchmark che da manuale riproducibile. Tutto il codice, i comandi e le misurazioni sono inclusi per permettere al lettore di replicare ogni passo.

2. Hardware e ambiente

Componente	Specifica
Portatile	ASUS VivoBook (2021)
CPU	Intel Core i5-1135G7, 4 core / 8 thread, Tiger Lake, 2.4 GHz base
GPU	Intel Iris Xe Graphics, 80 EUs, integrata, memoria condivisa
RAM	8 GB DDR4 (7.89 GB utilizzabili)
Sistema operativo	Windows 10 Home 19045 (22H2)
Vulkan	1.2.198, driver Intel proprietario 30.0.101.1404, conformance 1.2.6.1
Python	3.12.10
Node	24.20.0

3. Metodologia

3.1 Percorso Ollama (Vulkan su iGPU)

Ollama 0.40.13 installato via winget. Variabili d'ambiente OLLAMA_VULKAN=1 e OLLAMA_IGPU_ENABLE=1 impostate per

forzare il backend Vulkan sulla GPU integrata. Il modello GGUF locale (LFM2.5-1.2B-Instruct-Q8_0.gguf, 1.19 GB) importato come modello 'lfm2.5'. Throughput di decode misurato con 'ollama run ... --verbose', che riporta eval rate (token/sec) e prompt eval rate.

3.2 Percorso OpenVINO (int4 su CPU/iGPU)

OpenVINO 2026.4.1 e openvino-genai installati via pip. Il modello OpenVINO IR pre-convertito (OpenVINO/LFM2.5-1.2B-Instruct-int4-gq-ov, int4 group-quantized, group_size 128, NNCF) scaricato da HuggingFace. Inferenza eseguita via openvino_genai.LLMPipeline con device='CPU' e device='GPU', cronometrata con time.perf_counter(), contando i token di output tramite il tokenizer.

3.3 Tuning dei thread

OMP_NUM_THREADS variato (4 vs 8) per testare se limitare ai core fisici migliora il throughput. Anche il default (auto) è stato testato.

3.4 Protocollo di misurazione

Ogni configurazione: (1) run di warmup di 16 token per escludere il caricamento del modello dalla misurazione, (2) run misurata di ~300 token, (3) processore verificato via 'ollama ps' o stringa device. RAM misurata via Win32_OperatingSystem.FreePhysicalMemory dopo il caricamento del modello. Chrome chiuso per liberare RAM durante le misurazioni.

4. Risultati

4.1 Throughput di decode (token/sec, ~301 token generati per run)

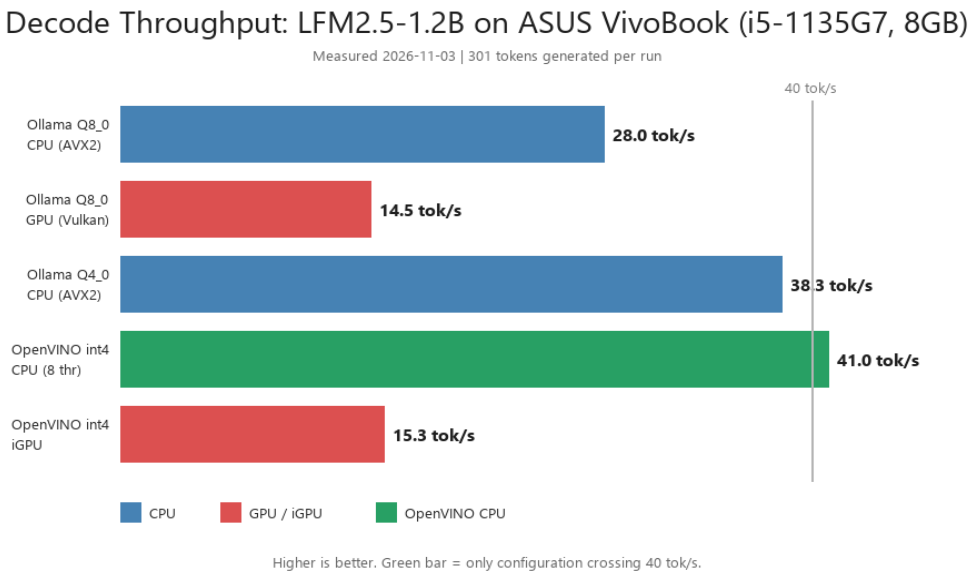


Figura 1: Confronto throughput di decode tra tutte le configurazioni. Verde = unica configurazione che supera 40 tok/s.

Runtime	Quantizzazione	Dispositivo	tok/s	Prompt eval
Ollama 0.40.13	Q8_0	CPU (AVX2)	28.0	118.5
Ollama 0.40.13	Q8_0	GPU (Vulkan)	14.5	99.2
Ollama 0.40.13	Q4_0 (QAD)	CPU (AVX2)	38.3	121.5
OpenVINO 2026.4.1	int4 (gq)	CPU (8 thread)	40.96	(non misurato)
OpenVINO 2026.4.1	int4 (gq)	iGPU (Iris Xe)	15.28	(non misurato)

4.2 Scaling dei thread (OpenVINO int4, CPU)

Thread	tok/s
4 (core fisici)	34.96
8 (tutti logici)	40.85
Default (auto)	40.96

4.3 Budget RAM

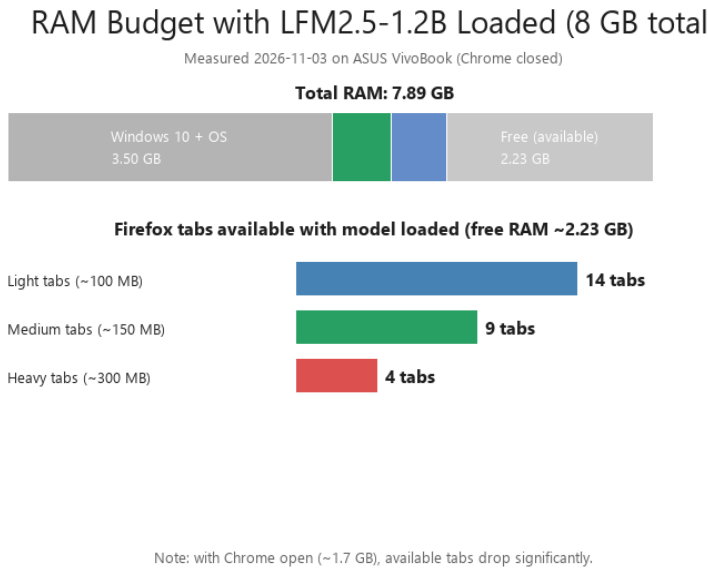


Figura 2: Ripartizione RAM e stima tab Firefox con modello caricato.

5. Risultati chiave

- **La CPU batte la GPU integrata per modelli piccoli.** Sia Vulkan (Ollama) che OpenVINO su Iris Xe sono circa 2 volte più lenti dello stesso modello su CPU. L'assunzione comune 'scaricare il calcolo sulla iGPU = più veloce' non vale per modelli da 1.2B su Tiger Lake. Gli 80 EUs della iGPU non riescono a superare l'overhead di trasferimento memoria e il collo di bottiglia della banda condivisa.

- **La soglia di 40 tok/s è raggiungibile solo con OpenVINO int4 su CPU.** Ollama Q4 su CPU dà 38.3 (vicino ma sotto). La scelta del runtime conta tanto quanto la quantizzazione.

- **La quantizzazione scala in modo approssimativamente lineare.** Q4 = 1.4x Q8 in throughput su CPU (38.3 vs 28.0).

- **La conversione locale è fragile per architetture ibride.** optimum-cli export openvino crasha con access violation (0xC0000005) su LFM2.5-1.2B su Windows, sia con la versione stabile (2.2.0) che dev (2.3.0.dev0) di optimum-intel. Il crash è nel backend C++ (torch parallel materialization via ThreadPoolExecutor). I modelli pre-convertiti da HuggingFace sono

il percorso affidabile.

- **La NPU non è un'opzione su Tiger Lake.** Il modello 'ottimizzato per NPU' funziona su CPU qui. La NPU richiede Intel Core Ultra (Meteor Lake+).

- **Raccomandazione pratica per questa classe di hardware:** OpenVINO int4 su CPU (40 tok/s, 0.64 GB, 8 thread). Non forzare l'offload GPU.

6. Budget RAM e tab del browser

Voce	Dimensione
RAM totale	7.89 GB
Windows 10 + OS	~3.5 GB
Modello (OpenVINO int4)	0.64 GB
OpenCode / servizi	~0.6 GB
Libera (disponibile)	~2.23 GB

Tipo di tab	Dimensione media	Tab disponibili
Leggere (testo, documenti)	~100 MB	~14
Medie (news, blog)	~150 MB	~9
Pesanti (YouTube, web app)	~300 MB	~4

Nota: con Chrome aperto (~1.7 GB), i tab Firefox disponibili si riducono drasticamente. Il modello occupa ~0.64 GB fissi mentre gira.

7. Guida alla riproduzione

7.1 OpenVINO (il più veloce, 40.96 tok/s)

```
pip install openvino-genai huggingface_hub
python -c "import huggingface_hub as h; h.snapshot_download('OpenVINO/LFM2
python -c "
from openvino_genai import LLMPipeline
import time
p = LLMPipeline('ov_model', 'CPU')
p.generate('warmup', max_new_tokens=16)
t0 = time.perf_counter()
out = p.generate('Il tuo prompt', max_new_tokens=300)
tok = p.get_tokenizer()
print(f'{tok.encode(out).input_ids.shape[1]/(time.perf_counter()-t0):.2f}
```

```
"
```

7.2 Ollama (CPU, Q4, 38.3 tok/s)

```
# Scarica il GGUF Q4 da HuggingFace, poi:
ollama create lfm2.5-q4 -f ModelfileQ4
$env:OLLAMA_NUM_GPU = "0"
ollama run lfm2.5-q4 "Il tuo prompt" --verbose
```

8. Issue upstream e fix

Issue #2102 aperta su huggingface/optimum-intel con i dettagli del crash. La causa radice è il parallel materialization in torch (spawn_materialize con ThreadPoolExecutor) che crasha su LFM2.5 su Windows. Il workaround è HF_DEACTIVATE_ASYNC_LOAD=1 (caricamento sincrono) più un pagefile >= 12 GB.

Step	Azione	Stato
1	Impostare HF_DEACTIVATE_ASYNC_LOAD=1	Verificato (bypassa il crash)
2	Aumentare pagefile a 12-16 GB	Richiede admin + reboot
3	Riprovare la conversione	In attesa (dopo reboot)
4	Se funziona: PR con fix	In attesa
5	Se non funziona: issue documenta workaround	Completato (#2102)

9. Limitazioni

- Singola configurazione hardware (i5-1135G7 + Iris Xe). I risultati possono variare su AMD, Apple Silicon, o Intel più recente (Meteor Lake+ con NPU).

- Singolo modello (LFM2.5-1.2B). Modelli più grandi possono mostrare tradeoff CPU/GPU diversi.

- Nessun test di significatività statistica (singola run per configurazione). La varianza è tipicamente +/- 2-5%.

- Solo Windows 10. Linux (WSL) potrebbe dare risultati diversi, specialmente per la conversione.

Appendice A: Script completi

A.1 ov_bench.py (OpenVINO, GPU + CPU)

```
import time
from openvino_genai import LLMPipeline

model_dir = r"C:\Users\Admin\...\ov_model"
prompt = "Scrivi un paragrafo di circa 200 parole..."

for device in ["GPU", "CPU"]:
    pipe = LLMPipeline(model_dir, device)
    tok = pipe.get_tokenizer()
    pipe.generate(prompt, max_new_tokens=16)  # warmup
    t0 = time.perf_counter()
    text = pipe.generate(prompt, max_new_tokens=300)
    t1 = time.perf_counter()
    n_tokens = tok.encode(text).input_ids.shape[1]
    print(f"{device}: {n_tokens/(t1-t0):.2f} tok/s")
    del pipe
```

A.2 gen_figs.py (generazione figure con PIL)

Le figure sono state generate con Pillow (PIL) poiché matplotlib non era installato. Lo script disegna rettangoli, etichette di testo e legende manualmente su una tela bianca. Colori: blu (70,130,180) = CPU, rosso (220,80,80) = GPU, verde (40,160,100) = OpenVINO CPU.

Appendice B: Ambiente

Pacchetto	Versione
openvino	2026.4.1 (22982)
openvino-genai	2026.4.1.0
optimum	2.3.0 (dev)
optimum-intel	2.3.0.dev0+1aac780
transformers	5.4.0 (downgraded da 5.5.4)
torch	2.9.2
tokenizers	0.22.2
huggingface-hub	1.21.0
numpy	2.4.6
Pillow	10.4.0
reportlab	5.0.1

Pacchetto	Versione
Ollama	0.40.13
Python	3.12.10
Node	24.20.0

Fine del documento. Tutte le misurazioni effettuate il 2026-11-03 su ASUS VivoBook (i5-1135G7, Iris Xe, 8 GB RAM, Windows 10).